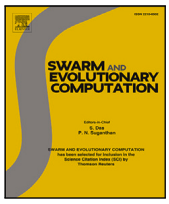
Swarm and Evolutionary Computation 104 (2026) 102392

Contents lists available at ScienceDirect

Swarm and Evolutionary Computation

journal homepage: www.elsevier.com/locate/swevo



A global feedback learning-based memetic algorithm for energy-aware scheduling in collaborative heterogeneous flexible job shops

Hongquan Qu, Shiliang Shao, Yunhong Xu, Maolin Cai, Yan Shi, Xiaomeng Tong *

School of Automation Science and Electrical Engineering, Beihang University (BUAA), Beijing, 100191, China

ARTICLE INFO

Keywords:
Industry 5.0
Energy-aware scheduling
Collaborative heterogeneous job shop scheduling
Memetic computing

ABSTRACT

Industry 5.0 emphasizes the collaboration between human and robot resources in perception, decision-making, and execution, forming a dynamic and complementary cooperative mechanism. However, multi-flexible resource collaboration significantly enlarges the solution space, which increases the difficulty of resource allocation and optimization. At the same time, the matching of jobs with machines and the switching of operational states, such as startup and shutdown, directly affect energy consumption, making energy savings and cost reduction rigid requirements. To address these issues, a collaborative heterogeneous flexible job shop scheduling (CHFJS) model is formulated, with the primary objectives of minimizing makespan and energy consumption. Subsequently, a memetic algorithm based on global feedback learning (GFLMA) is proposed to solve the CHFJS problem. A total of 12 neighborhood structures are designed, and a Bayesian inference and weighting-based local search strategy is established. Additionally, energy-saving operators are specifically designed to address the problem characteristics. Finally, extensive experiments on instances of various scales are conducted to validate the effectiveness of GFLMA. The results demonstrate that the proposed algorithm outperforms state-of-the-art algorithms in more than 70% of the instances, confirming the superiority of GFLMA.

1. Introduction

With the advancement of economic globalization, the customer base of manufacturing enterprises has become increasingly diverse. Therefore, manufacturing enterprises require high production flexibility to adapt to the heterogeneous demands of various industries rapidly. In flexible manufacturing environments, each machine can process multiple jobs, but due to performance differences, processing times vary across machines. Scheduling in such an environment is defined as the classical flexible job shop scheduling (FJS) [1]. Despite continuous technological advancements, individual machines remain limited in capability and cannot process all types of jobs. In practice, operations can only be processed by a subset of machines. This heterogeneity in machine processing capabilities gives rise to the heterogeneous flexible job shop scheduling (HFJS) [2]. Furthermore, the manufacturing process usually involves various auxiliary collaborative units, such as workers or robots. In traditional collaboration modes, these units are required to track the processing status throughout the entire cycle, resulting in continuous occupation. However, significant breakthroughs in intelligent manufacturing technologies are promoting the construction of highly automated and sustainable smart factories [3]. For instance, benefiting from intelligent fixtures and anomaly monitoring systems, continuous occupation of humans or robots during job processing is no longer necessary [4]. Collaborative units are only required to participate in the pre-processing stage and can be released to handle other tasks during the automated processing. This new collaboration mode breaks traditional occupation constraints. Therefore, based on the heterogeneity of machine capabilities and the non-continuous occupation characteristics of collaborative units, we propose the collaborative heterogeneous flexible job shop scheduling (CHFJS). This introduces complex spatiotemporal coupling constraints to the HFJS, leading to higher computational complexity.

Within the context of Industry 5.0, collaborative shop scheduling has attracted increasing attention. For instance, Arviv et al. [5] investigated optimal strategies for dual-robot cooperation in flow shop scheduling with job transfers, showing that dual-robot collaboration achieved the best results. Sadik et al. [6] addressed flow shop scheduling with human–robot cooperation using a two-stage Johnson's algorithm to determine the optimal sequence of jobs. Kinast et al. [7] extended classical job shop scheduling by incorporating collaborative robots into workstation assignments and developed a hybrid genetic algorithm to solve the problem. Wang et al. [4] proposed a stochastic

* Corresponding author.
E-mail address: tongxiaomeng@buaa.edu.cn (X. Tong).

https://doi.org/10.1016/j.swevo.2026.102392
Received 30 September 2025; Received in revised form 8 February 2026; Accepted 7 April 2026
Available online 8 April 2026

flow shop scheduling problem under human–robot collaboration, formulating a joint chance-constrained model based on the Cobb–Douglas function to achieve efficient resource utilization. Vermin et al. [8] introduced a bi-objective scheduling model for collaborative order-picking systems that accounts for worker fatigue and productivity, with experimental results demonstrating its potential for improving scheduling performance. Lu et al. [9] were the first to apply human–robot collaboration in welding shop scheduling and proposed a memetic algorithm combining multiple neighborhood search strategies, yielding significant gains in productivity and energy efficiency. Yu et al. [10] investigated a distributed welding shop scheduling problem with fuzzy processing times under human–robot collaboration in the context of Industry 5.0, and proposed a self-learning memetic algorithm for efficient and low-energy scheduling optimization. Although these studies lay a solid foundation for collaborative shop scheduling, most research has focused on flow shops or classical job shops. Progress has been made in areas such as human–robot and dual-robot collaboration. However, the collaborative scheduling problem involving multi-flexible auxiliary resource coupling in flexible job shop environments has not yet been explored. Considering the rapid technological development, research on CHFJS provides crucial theoretical support for flexible intelligent manufacturing. Furthermore, with the growing global emphasis on environmental protection, energy consumption has become a critical concern in many industrial domains [11]. Designing energy-saving strategies tailored to CHFJS thus holds significant value.

Classical HFJS poses two subproblems: optimizing the processing sequence of operations among different jobs and selecting a machine from the available subset for each operation. Existing research has proposed five primary methods to address these challenges: (1) **Exact Solution Methods**: Methods like linear programming [12,13] guarantee finding the global optimal solution. However, they suffer from excessively long solution times and massive memory consumption for even moderately sized problems. (2) **Heuristic Algorithms**: Simple in structure and computationally inexpensive, greedy algorithms [14] and priority rules [15] have low computational complexity. Nevertheless, their performance boundaries are difficult to analyze, and they lack global optimization capabilities [16]. (3) **Metaheuristic Algorithms**: Such as genetic algorithms and particle swarm optimization [17–19], are well suited for combinatorial optimization problems and exhibit strong global search capability, but they are prone to being trapped in local optima. (4) **Deep Reinforcement Learning**: Methods based on deep Q-learning [20] and graph neural networks [21] require extensive training and large datasets. They also have limitations in terms of generalization and multi-objective analysis. 5) **Hybrid Algorithms**: Memetic algorithms [22,23] integrate the global exploration of metaheuristics with the local search of heuristic rules, allowing them to achieve near-optimal solutions with strong problem adaptability. They have become a mainstream method for solving job shop scheduling problems [24].

The memetic framework, which combines evolutionary exploration with local search, has proven effective in finding high-quality solutions for classical HFJS [25]. However, existing experience-driven memetic frameworks exhibit low efficiency when applied to CHFJS, mainly due to three reasons. First, the introduction of collaborative units grants CHFJS greater flexibility, leading to a significantly larger search space. Consequently, an experience-dependent memetic framework struggles to explore a broader range of potential objective space. To address this, we have designed a regional initialization method that distributes initial solutions across a wider solution space, enhancing global exploration and facilitating regional feedback and environmental selection. Second, in contrast to classical FJS, CHFJS not only requires adjusting operation nodes on the critical path but must also simultaneously handle four mutually coupled subproblems: machine assignment, operation sequence, collaborative unit assignment, and collaborative unit auxiliary sequence. Original critical path optimization methods are therefore difficult to apply. To overcome this, our work conducts an in-depth analysis of the CHFJS critical path structure and, based on this, designs a variable neighborhood search (VNS) with 12 neighborhood structures to enhance the joint optimization of operations and collaborative units. Finally, the selection method for neighborhood structure operators has become a key factor limiting performance. Existing operator selection strategies can be broadly categorized into random selection [26], exhaustive selection [27], hierarchical selection [28], and confidence-based selection [29]. However, these strategies suffer from three major limitations. First, random or exhaustive selection lacks feedback guidance, which often results in unguided exploration and consequently low search efficiency and large performance variance across runs. Second, hierarchical selection strategies based on historical experience, despite using feedback, are constrained by their periodic update mechanism, which causes delayed guidance information; consequently, operator selection cannot respond in real time to changes in the current population's search status. Third, confidence-based methods (e.g., reinforcement learning approaches), although they exploit feedback information, are highly prone to weight accumulation bias. Such methods tend to over-rely on operators that perform well in the early stage of the search, while neglecting other potentially effective operators. Moreover, they typically require a large number of training samples, leading to high computational cost [30]. The related studies are summarized in Table 1. Based on the above limitations, to address this issue, we introduce a wisdom of crowds (WoC) decision model. Compared to traditional strategies, the WoC model utilizes global feedback information to dynamically learn and select neighborhood operators, effectively correcting the bias caused by a single evaluation perspective while avoiding the expensive training costs. Through these improvements, the proposed global feedback learning-based improved memetic framework maintains broad exploration while also deeply optimizing complex critical paths and diverse neighborhood operators, thereby significantly enhancing the solution efficiency and quality for CHFJS.

The WoC effect reveals a crucial phenomenon: cooperative groups can make wiser judgments more efficiently than individuals. Numerous studies have shown that this theory can effectively solve complex decision-making problems by ensuring individual independence and aggregating diverse opinions [31]. Due to its advantages, it is currently widely used in various decision-making scenarios, including British general election predictions [32], business investment strategy selection [33], affective decision-making [34], and biological analysis [35], demonstrating strong adaptability and influence in practical applications. Research results on problems such as the traveling salesman problem [36], sudoku [37], and combinatorial optimization [38] using WoC algorithms also demonstrate its effectiveness for NP-hard problems. Relevant surveys indicate that WoC has not yet been applied to job shop scheduling problems. Based on these findings and considering the growing number of neighborhood structure operators, we designed a WoC-based decision model to implement an adaptive selection mechanism.

This work proposes a global feedback learning-based memetic algorithm (GFLMA), and its main contributions are summarized as follows.

(1) **Problem Extension**: Smart manufacturing-based CHFJS is studied for the first time. It considers a scenario of collaborative production between different resources, aiming to minimize the makespan and total energy consumption simultaneously.

(2) **Global Feedback Learning Framework**: Both global evolution and local search adopt a feedback learning mechanism. To address the vast search space of CHFJS, the global evolution process ensures population diversity and regional division during initialization. By learning the regional lineage of elite individuals, the search space is focused on areas with potentially high-quality solutions. Furthermore, the local search adaptively selects neighborhood structure operators based on crowdsourced feedback.

Table 1
Overview of related studies.

References	Collaboration scenario	Objective number	Energy consumption	Feedback method	Scheduling scenario	Algorithm abbreviation
Wang et al. [4]	✓	1	×	–	A	ESH
Sadik et al. [6]	✓	1	×	–	A	JA
Vermin et al. [8]	✓	2	×	–	D	SCIP
Lu et al. [9]	✓	2	✓	–	C	PMA
Yu et al. [10]	×	1	×	–	A	KBIG
Yu et al. [13]	✓	2	✓	–	C	SLMA
Xu et al. [18]	×	3	✓	–	B	QPSO
Lu et al. [22]	×	2	✓	②	A	CMOA
Chang et al. [25]	×	3	×	–	B	RL-MOMA
Huang et al. [26]	×	2	✓	①	A	BRCE
Yang et al. [27]	×	2	✓	②	B	DBMA
Huang et al. [28]	×	2	✓	③	B	EMAH
Li et al. [29]	×	2	✓	④	B	DQCE
This Study	✓	2	✓	⑤	B	GFLMA

Note: ①: random selection, ②: exhaustive selection, ③: hierarchical selection, ④: confidence-based selection, ⑤: WoC-based selection; A: flow shop scheduling, B: flexible job shop scheduling, C: welding shop scheduling, D: job shop scheduling.

(3) **Neighborhood Structure Operators**: In the CHFJS solution, we analyze the critical path using a disjunctive graph model. To address the bottleneck effect of collaborative units, we design 12 neighborhood structure operators. This study is the first to expand the number of neighborhood structure operators to more than 10, providing a richer set of strategies for local search in complex job shop scheduling problems.

(4) **Multi-Operator Selection**: Addressing the multi-neighborhood structure operator selection problem, we propose a novel WoC decision model. This model combines Bayesian inference with dynamic weighting to estimate the success rate of optimization. It learns dynamically through feedback during the group optimization process, allocating selection probabilities to the 12 neighborhood structure operators. We introduce an aggregation table and a success rate ranking mechanism to mitigate the impact of historical accumulation, thereby preventing inefficient exploration with low-performing operators.

The remainder of this paper is organized as follows. Section 2 describes the mathematical model of CHFJS. Section 3 explains the framework and key components of GFLMA. Section 4 reports the numerical study results. Section 5 concludes this work and provides an outlook for future work.

2. Problem description and mathematical model

2.1. Problem description

The specific interactive relationship between collaborative units and machines within the job processing workflow is illustrated in Fig. 1. In the "Before processing" stage, a collaborative unit (e.g., worker C or robot C') is responsible for assisting the job J and the fixture to complete pre-processing tasks such as loading and positioning. Subsequently, in the "During processing" stage, the machine M takes over the job and independently executes the automated processing, during which the collaborative unit is no longer involved. Therefore, CHFJS can be described as follows: before each job is processed, it requires assistance from collaborative units. A total of n jobs are assigned to m machines, and each job contains n_i operations. The same operation $O_{i,j}$ can be processed by multiple optional machines, with different processing times on different machines.

CHFJS needs to solve the following four subproblems: (1) machine assignment: assigning machines for operations; (2) operation sequencing: determining the processing order of operations among different jobs; (3) collaborative unit assignment: assigning collaborative units to operations; (4) collaborative unit sequencing: determining the working order of collaborative units. To simplify the problem, the following assumptions are made:

- At time zero, all machines and collaborative units are available.
- Collaborative units are considered auxiliary to operations and corresponding machines.
- Each operation requires two collaborative units.
- Once started, operations cannot be interrupted.
- At the same time, each machine and collaborative unit can only process one task.
- Transportation time and dynamic events are not considered.

2.2. MILP model

2.2.1. Indices

(1) Indices

i: index of jobs, $i = 1, \dots, n$.
j: index of operations, $j = 1, \dots, n_i$.
k: index of machines, $k = 1, \dots, m$.
t: index of collaborative units, $t = 1, \dots, r$.
p: index of positions on machine M_k, $p \in P_k$.
p': index of positions of collaborative unit t on machine M_k, $p' \in P_{t,k}$.

(2) Sets

I: set of jobs.
J_i: set of operations of job i.
M: set of heterogeneous machines.
C: set of all collaborative units, $C = C^1 \cup C^2$.
C^1: set of the first type of collaborative units (workers).
C^2: set of the second type of collaborative units (robots).
$M_{i,j}$: set of machines available for the jth operation of the ith job.
P_k: set of processing positions on machine M_k.
$P_{t,k}$: set of positions of collaborative unit t on machine M_k.

(3) Parameters

n: total number of jobs.
m: total number of machines.
r: total number of collaborative units.
n_i: total number of operations of job i.
e_r: the energy consumption for one restart cycle.
$O_{i,j}$: the jth operation of the ith job.
M_k: the kth machine.
C_t: the tth collaborative unit.
$T_{i,j,k}$: the processing time of $O_{i,j}$ on machine M_k.
$T_{t,i,j,k}$: the working time of the collaborative unit before $T_{i,j,k}$ on machine M_k.
T_{th}: the machine shutdown time threshold.
N_P: the processing power of all machines.
N_I: the idle power of all machines.
N_C: the power of robotic collaborative units ($t \in C^2$).
L: a sufficiently large positive number.

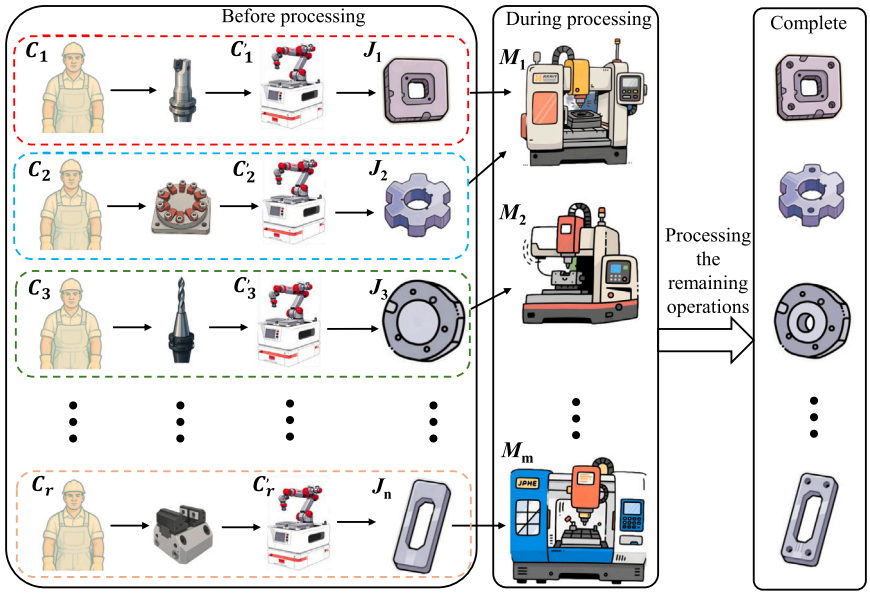


Fig. 1. Layout of CHFJS.

(4) Decision Variables

$C_{p,k}$: the finish time of the operation assigned to position p on machine M_k.

$S_{p,k}$: the start time of the operation assigned to position p on machine M_k.

$S_{i,j,k}$: the start processing time of $O_{i,j}$ on machine M_k.

$F_{i,j,k}$: the finish processing time of $O_{i,j}$ on machine M_k.

F_k: the max finish time on machine M_k.

$S_{t,i,j,k}$: the start working time of collaborative unit C_t for $O_{i,j}$ on machine M_k.

$F_{t,i,j,k}$: the finish working time of collaborative unit C_t for $O_{i,j}$ on machine M_k.

E_P: the total energy consumption during the processing period.

E_I: the total energy consumption during the idle period.

E_R: the total restart energy consumption.

E_C: the total energy consumption of collaborative units (robots).

TEC: total energy consumption.

$C_{\max}$: the maximum completion time (makespan).

$X_{i,j,k,p}$: binary variable; equals 1 if $O_{i,j}$ is assigned to position p on machine M_k, 0 otherwise.

$B_{t,i,j,k,p'}$: binary variable; equals 1 if C_t is assigned to perform collaborative work for $O_{i,j}$ on machine M_k at position p', 0 otherwise.

The mathematical model includes the following objective functions and constraints:

$$\min F = [C_{\max}, TEC] \tag{1}$$

subject to:

$$\sum_{k\in M_{i,j}} \sum_{p\in P_k} X_{i,j,k,p} = 1, \quad \forall i \in I, j \in J_i \tag{2}$$

$$\sum_{i\in I} \sum_{j\in J_i} X_{i,j,k,p} \le 1, \quad \forall k \in M, p \in P_k \tag{3}$$

$$\sum_{t\in C} \sum_{k\in M_{i,j}} \sum_{p'\in P_{t,k}} B_{t,i,j,k,p'} = 2, \quad \forall i \in I, j \in J_i \tag{4}$$

$$\sum_{p'\in P_{t,k}} B_{t,i,j,k,p'} \le \sum_{p\in P_k} X_{i,j,k,p}, \quad \forall i \in I, j \in J_i, t \in C, k \in M_{i,j} \tag{5}$$

$$S_{i,j+1,k'} \ge F_{i,j,k} - L\cdot\left(2 - \sum_{p\in P_k} X_{i,j,k,p} - \sum_{q\in P_{k'}} X_{i,j+1,k',q}\right), \quad \forall i \in I, j \in J_i, j < |J_i| \tag{6}$$

$$F_{i,j,k} = S_{i,j,k} + T_{i,j,k} \cdot \sum_{p\in P_k} X_{i,j,k,p}, \quad \forall i \in I, j \in J_i, k \in M_{i,j} \tag{7}$$

$$F_{t,i,j,k} = S_{t,i,j,k} + T_{t,i,j,k} \cdot \sum_{p'\in P_{t,k}} B_{t,i,j,k,p'}, \quad \forall i \in I, j \in J_i, k \in M_{i,j}, t \in C \tag{8}$$

$$S_{t,i',j',k'} \ge F_{t,i,j,k} - L\cdot\left(2 - \sum_{p'\in P_{t,k}} B_{t,i,j,k,p'} - \sum_{q'\in P_{t,k}} B_{t,i',j',k',q'}\right), \quad \forall t \in C, (i,j) < (i',j') \tag{9}$$

$$S_{i,j,k} \ge F_{t,i,j,k} - L\cdot\left(1 - \sum_{p'\in P_{t,k}} B_{t,i,j,k,p'}\right), \quad \forall i \in I, j \in J_i, k \in M_{i,j}, t \in C \tag{10}$$

$$F_k \ge F_{i,j,k} - L\cdot\left(1 - \sum_{p\in P_k} X_{i,j,k,p}\right), \quad \forall k \in M, i \in I, j \in J_i \tag{11}$$

$$E_P = \sum_{i\in I} \sum_{j\in J_i} \sum_{k\in M_{i,j}} \sum_{p\in P_k} T_{i,j,k} \cdot N_P \cdot X_{i,j,k,p} \tag{12}$$

$$E_I = \sum_{k\in M} \sum_{p\in P_k} \left(S_{p+1,k} - C_{p,k}\right) \cdot N_I \cdot \mathbb{1}\left(\left(S_{p+1,k} - C_{p,k}\right) < T_{th}\right) \tag{13}$$

$$E_R = \sum_{k\in M} \sum_{p\in P_k} e_r \cdot \mathbb{1}\left(\left(S_{p+1,k} - C_{p,k}\right) \ge T_{th}\right) \tag{14}$$

$$E_C = \sum_{t\in C^2} \sum_{i\in I} \sum_{j\in J_i} \sum_{k\in M_{i,j}} \sum_{p'\in P_{t,k}} T_{t,i,j,k} \cdot N_C \cdot B_{t,i,j,k,p'} \tag{15}$$

$$C_{\max} \ge F_{i,j,k} - L\cdot\left(1 - \sum_{p\in P_k} X_{i,j,k,p}\right), \quad \forall i \in I, j \in J_i, k \in M_{i,j} \tag{16}$$

$$TEC = E_P + E_I + E_R + E_C \tag{17}$$

$$S_{i,j,k}, F_{i,j,k}, S_{t,i,j,k}, F_{t,i,j,k}, F_k \ge 0 \tag{18}$$

Eq. (1) defines the optimization objective of CHFJS. Eq. (2) ensures that each operation must and can only be assigned once to a position on a machine. Eq. (3) specifies that a machine can process only one operation at the same position and time. Eq. (4) constrains the number of collaborative units required for each operation. Eq. (5) ensures that

the assignment of collaborative units is consistent with the machine assignment of operations. Eq. (6) ensures the processing sequence of operations. Eqs. (7) and (8) respectively define the processing time of operations and the working time of collaborative units. Eq. (9) describes that the working time intervals of the same cooperative unit do not overlap. Eq. (10) stipulates that the processing time of an operation must be after the completion time of its assigned collaborative units. Eq. (11) defines the maximum finish time on each machine. Eq. (12) to (15) formulate the energy consumption calculations, where $\mathbf{1}(\cdot)$ is the indicator function that equals 1 if the condition in the parenthesis is true, and 0 otherwise. Eqs. (16) and (17) provide the formulas for calculating the total energy consumption and the total makespan. Eq. (18) constrains the non-negativity of the decision variables.

3. Our approach: GFLMA

3.1. Motivation

The motivation for using a global feedback learning evolutionary framework (GFLE) to solve the proposed problem is as follows: (1) CHFJS contains not only job assignment and sequencing but also constraints among collaborative units, which results in a huge solution space. Under limited computational resources, it is necessary to steer the initial population to cover broader regions of the solution space at the early stage. Therefore, the initial population is divided into regions, and regional ancestry is preserved during iterations. Subsequent population updating is performed based on feedback and learning. (2) Since CHFJS is studied for the first time, there are no existing neighborhood structures for this problem. Therefore, we first analyze the critical path and nodes of CHFJS and then design 12 efficient local search operators based on this analysis, thereby accelerating the convergence of the population. (3) Confidence-based selection models for local search operators are usually affected by cumulative historical results, which may cause effective operators to be ignored in the early stage. The WoC algorithm, based on Bayesian rules, can make faster decisions, selecting high-efficiency operators and avoiding wasting resources on inefficient ones. (4) In collaborative scheduling, idle time windows between machines also contain collaborative unit working time, which makes the conditions for inserting operations into idle windows more stringent. Currently, no suitable energy-saving strategy exists for this problem. Therefore, an inverse full-active scheduling strategy is designed, which delays the processing of collaborative jobs by backward searching, thereby optimizing energy consumption on other machines and reducing TEC.

3.2. Global feedback learning evolutionary framework

The core idea of GFLE is that each evolutionary stage learns from feedback metrics. As shown in Fig. 2, GFLE connects four key stages through a closed-loop feedback mechanism (see Fig. S1 in the supplementary material for implementation details). (1) Region-based initialization: Generate the initial population P_0 and label individuals with regional ancestry according to their distribution characteristics. (2) Global evolutionary search: Use the regional-ancestry feedback signals to guide intergenerational selection, and perform crossover and mutation to generate offspring. (3) Local search: Identify optimization nodes via critical-path analysis, and achieve adaptive operator selection using operator-performance feedback. (4) Environmental selection and closed loop: Select the next-generation population P_{t+1} via non-dominated sorting. Meanwhile, the regional distribution of elite individuals and the operator contributions are summarized to generate feedback signals that are sent back to the search stage. This process iterates until the Pareto-optimal solutions are obtained.

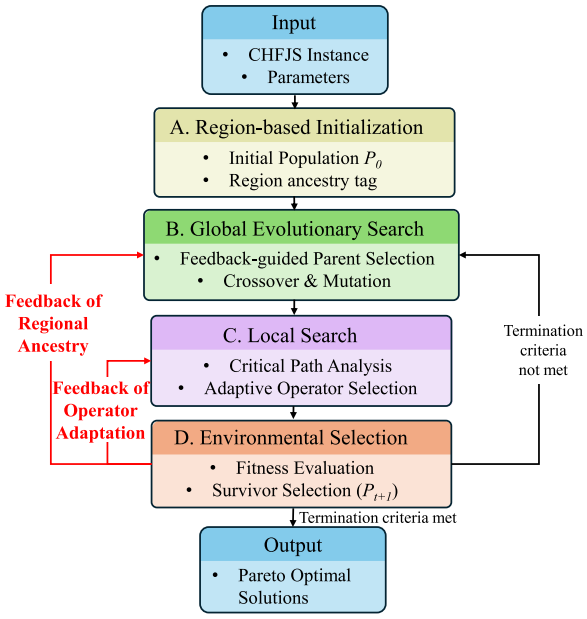


Fig. 2. Evolutionary process based on global feedback learning.

3.3. Encoding and decoding

Sequence encoding simplifies computation and is suitable for narrowing the search space [39]. Therefore, a multi-layer sequence structure is adopted to represent scheduling solutions. As shown in Fig. 3, each $O_{i,j}$ requires one machine and two collaborative units for processing. The scheduling solution comprises three jobs, eight operations, four machines, and three types of collaborative units. The encoding and decoding methods are explained as follows.

Encoding: The encoding consists of the operation sequence (OS), machine sequence (MS), and collaborative unit sequence (CS), where the number of operations determines the number of CS layers. Accordingly, the encoding structure is designed as four parallel sequences of equal length. In OS, each element represents a job ID, and the number of occurrences of that ID corresponds to the operation number. For example, if the fourth element in OS is 3, it refers to the second operation of job 3, denoted as $O_{3,2}$, since the first occurrence of job 3 appears at the third position. MS and CS adopt the same indexing rule, with each position corresponding to the machine or collaborative unit assigned to the operation. For instance, the third value in MS indicates that operation $O_{1,3}$ is processed on machine M_3.

Decoding: Decoding proceeds by sequentially extracting each operation $O_{i,j}$ from OS. Based on the same index, the corresponding CS values provide collaborative unit information, and the MS value specifies the processing machine M_k. The operation $O_{i,j}$, together with its collaborative units $C_{1,i,j}$ and $C_{2,i,j}$, is then scheduled on M_k under the given constraints. Once all operations are completed, the maximum completion time, known as the makespan, is calculated, and the TEC is derived from processing times and machine idle periods.

3.4. Regional initialization

To maintain diversity, scheduling problems typically employ random initialization [40], whereas some studies utilize problem-based initialization [41]. However, the importance of initialization in scheduling problems has not been widely discussed. Therefore, a regional initialization method is designed, as shown in Fig. 4. The procedure is as follows.

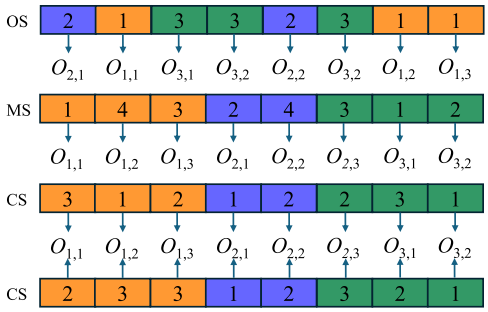


Fig. 3. Example of encoding solution for CHFJS.

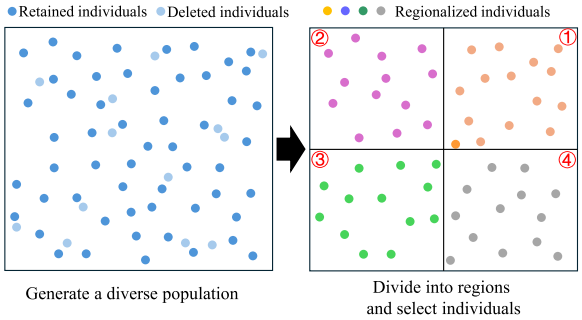


Fig. 4. Example of regional initialization.

(1) Generate a population larger than pop_i using multiple initialization methods (Latin hypercube, random, problem-based, and minimum load initialization).

(2) Decode all individuals, calculate makespan and TEC, and divide the solution space into four regions by their medians.

(3) For each region, calculate crowding distances, retain high-ranking individuals, and mark their ancestry, called regional ancestry, to guide parent selection.

To analyze the three initialization methods, experiments were conducted on benchmark datasets [42,43] using the classical algorithms MOEA/D [44] and NSGA-II [45]. The results presented in the supplementary materials (Table S-1 and Table S-2) demonstrate that regional initialization can significantly improve the convergence performance and overall solution quality for both MOEA/D and NSGA-II, outperforming traditional random initialization and problem-based initialization methods. It is worth noting that this performance improvement exhibits a consistent trend across different algorithms and does not alter their relative performance rankings. This indicates that regional initialization mainly enhances the quality of the initial population distribution in the objective space, thereby providing a more sufficient and balanced starting point for subsequent search, rather than exerting a targeted enhancement on the evolutionary operators or search mechanisms of specific algorithms.

Based on the above analysis, regional initialization is uniformly applied to all comparative algorithms in the subsequent experiments. The purpose is to minimize the potential bias caused by differences in initial population quality, thereby enabling a fairer evaluation of the intrinsic evolutionary mechanisms of different algorithms under a higher and more consistent baseline.

3.5. Transgenerational environmental selection

Environmental selection is the primary strategy for producing elite solutions [45]. Inspired by cross-generation selection [46], a method based on regional ancestry is designed. Elite individuals are selected from offspring and previous generations. To prevent premature convergence, individuals similar in ancestry but not yet selected are also retained. Algorithm 1 illustrates the procedure. In the PBX and Mutation function, crossover is executed with a probability of p_c. Specifically, the operation sequence (OS) is recombined using the position-based crossover (PBX) operator. In contrast, the machine sequence (MS) and collaborative unit sequence (CS) are recombined using the uniform crossover (UX) operator. Detailed descriptions of PBX and UX are provided in the supplementary materials. After crossover, mutation is applied with probability p_m. In OS, two operations are randomly swapped, while in MS and CS, two positions are randomly selected, and their corresponding M_k and $C_{t,i,j}$ values are exchanged.

Algorithm 1: Transgenerational environmental selection based on regional ancestry

Input: Current iteration number (t), parent population ($parentPop$), crossover probability (p_c), mutation probability (p_m)
Output: Population (Pop)

```
if t = 1 then
    offspringPop ← PBXandMutation(p_c, p_m, parentPop);
    parentPop ← Copy(offspringPop);
    preOffspringPop ← Copy(offspringPop);
    Pop ← Copy(offspringPop);
else
    offspringPop ← PBXandMutation(p_c, p_m, parentPop);
    elitePop ← FastNonDominatedSort(offspringPop);
    regionalAncestry ← RegionalAncestryDef(elitePop);
    regionalPop ← CalcRegionalProportion(regionalAncestry, preOffspringPop);
    mergedPop ← Merge(offspringPop, preOffspringPop);
    Pop ← Merge(mergedPop, regionalPop);
    preOffspringPop ← Copy(offspringPop);
```

3.6. Hierarchical critical path analysis

Critical path analysis is essential for optimizing the makespan and for constructing local search [47]. In decoding, the makespan is the maximum completion time, and the critical path is the longest path that determines the makespan. In FJSP, a disjunctive graph is used to extract the longest path from start to end, where each operation on it is a critical operation. Since the traditional disjunctive graph includes only operations, a modular disjunctive graph with collaborative units is designed. It is defined as:

$$G = (V; A \cup E) \tag{19}$$

where: $V = V_o \cup V_c$; $A = A_o \cup A_c \cup A_{oc}$, $E = E_o \cup E_c$. Here, the sets are defined as follows. A_o: order of operations in jobs. A_c: order of collaborative units. A_{oc}: cooperation between collaborative units and operations. Moreover, E_o: competition of operations on machines. E_c: competition of collaborative units on machines. As shown in Fig. 5, the steps are: (1) Decompose operations and collaborative units, identify bottlenecks if connected. (2) Map the Gantt chart to a disjunctive graph. Apply classical critical path analysis to operations [48]. For collaborative units without bottlenecks, treat them as operations. (3) Extract nodes from two critical paths, reduce search scope, and compute critical paths in reverse. Define blocks as sets of consecutive nodes on the same machine.

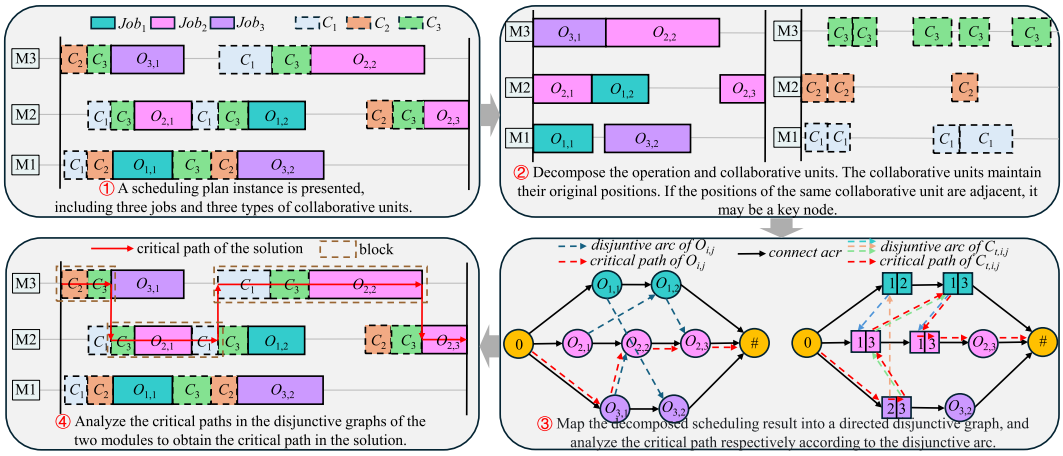


Fig. 5. Critical path analysis procedure.

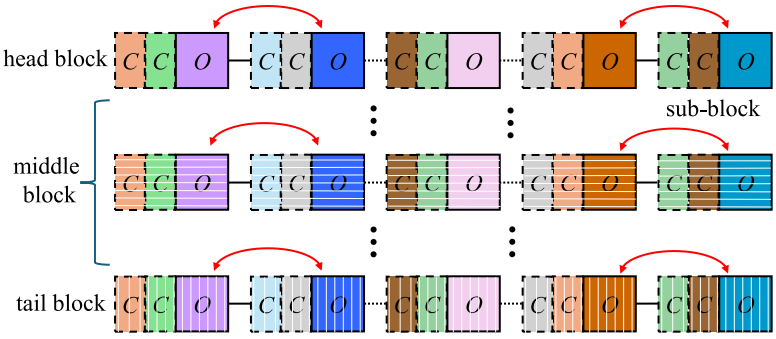


Fig. 6. Example diagram of the N1 structure.

3.7. Problem-based local search

Local search enhances solution quality by reordering critical nodes [49]. The classical VNS adjusts operation nodes, but CHFJS involves both operations and collaborative units. Therefore, a problem-based VNS with 12 neighborhood structures is designed. Specifically, N1–N6 adjust operation nodes, N7–N10 adjust collaborative unit nodes (considering only cases where collaborative units are connected in the disjunctive graph), and N11–N12 perform composite adjustments. The details are as follows.

(1) N1 (Classical N5 Structure): The classical N5 structure forms the basis of all subsequent neighborhoods [50]. Inspired by it, the N1 structure is designed. As shown in Fig. 6, the critical path typically consists of multiple blocks: the first block is the head block, the last block is the tail block, and the others are middle blocks. A complete collaborative block is termed a sub-block. The N1 operation swaps the first two and the last two operation nodes within each block.

(2) N2 (Internal Node Shift): To completely break the scheduling order of operation nodes within a block, internal operation nodes are moved close to the front of the first node or the end of the last node.

(3) N3 (End Node Shift): An internal node is randomly selected, and the N2 movement is applied. In addition, the first and last nodes are shifted into internal positions, with the constraint that the first node is placed after the original last node.

(4) N4 (Longest Block Node Exchange): Since adjusting the longest block of the critical path has a higher probability of yielding better solutions, two operation nodes are randomly selected within the longest block and swapped.

(5) N5 (Intra- and Inter-Block Node Exchange): Operation nodes inside a block are swapped with nodes outside the block, which may reduce block length [51]. Specifically, two nodes within a block are exchanged with nodes outside the block but on the same machine.

(6) N6 (Inter-Machine Node Exchange): Processing times and constraints differ across machines. Therefore, two blocks on different machines are selected, and one node from each block is swapped.

(7) N7 (Sub-Block Internal-External Node Exchange): When collaborative units are connected in the disjunctive graph, their working time often becomes a bottleneck. To disrupt this relationship, two collaborative unit nodes belonging to different sub-blocks within the same block are selected and swapped.

(8) N8 (Sub-Block External Node Machine Exchange): Collaborative unit nodes outside sub-blocks are identified across different machines, with the requirement that their working times are far apart. Two such nodes are then swapped.

(9) N9 (Sub-Block and Block External Node Exchange): On each machine, two collaborative unit nodes outside sub-blocks and two collaborative unit nodes outside blocks are randomly selected, and the four nodes are swapped.

(10) N10 (Node Mutation): Unlike the global evolution phase, where mutations are random, in local search, targeted mutation is introduced. The longest-working collaborative unit node in the longest block is replaced by another collaborative unit node.

(11) N11 (Sub-Block Shift of Operation Nodes): The previous neighborhoods adjust nodes individually, but heterogeneous flexibility may cause infeasible solutions. Composite optimization is a commonly used strategy in industry [52], thus a neighborhood structure based on sub-block interchange was designed. To improve success rates, a heuristic roulette-wheel mechanism is used to select a sub-block: different middle blocks are randomly selected, and the scheduling frequency of each job's operations before this block is counted as $S(J_i) = \sum_{m=1}^{k} \sum_{t=0}^{t'} J_i$. Jobs scheduled more frequently are more likely to create bottlenecks.

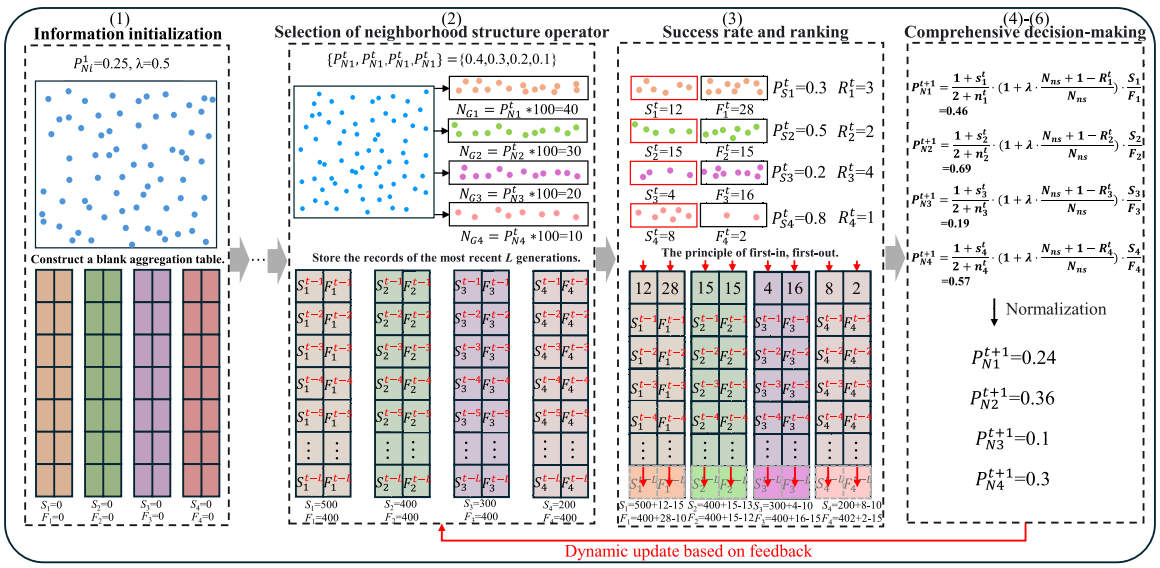


Fig. 7. WoC feedback learning decision model.

The selection probability is computed as $P(B_i) = \frac{S(J_i)}{\sum_{B_j \in B} S(J_j)}$, and the chosen sub-block is moved to the end of its block or the next block.

(12) N12 (Sub-Block Shift of Collaborative Unit Nodes): Similarly, the scheduling frequency of collaborative units is calculated, and roulette-wheel selection is applied to choose two collaborative units from different sub-blocks. The corresponding sub-blocks are then shifted sequentially to the end of the current or the next block.

3.8. WoC decision model

Although knowledge-based neighborhood structures are effective, efficient operator selection among multiple operators for CHFJS remains a key obstacle to their effective integration [53]. Traditional adaptive selection strategies, due to the cumulative effect of historical rewards, often struggle to respond quickly in dynamic search environments and are prone to falling into local operator preferences. To address this issue, based on the WoC theory, an adaptive selection mechanism that balances individual independence and group consensus is designed. First, each individual dynamically selects from 12 neighborhood operators according to selection probabilities. Then, a beta distribution is adopted as the prior for the optimization success probability of each operator, and Bayesian updating is applied to estimate the expected optimization success. Finally, by combining success rates with ranking-based weighting, the weights of high-performing operators are reinforced, enabling feedback learning to generate the selection probabilities of the next generation population. WoC ensures that the collective decision of operators reflects the wisdom of the population by extracting the best operators at the population level, allowing rapid adaptation to the current optimization stage. Fig. 7 illustrates the feedback learning process of this model. Suppose the inputs are a population consisting of 100 individuals, four candidate operators, and the initialized parameters. At generation t, if the selection probabilities of the four operators are $P_N^t = \{0.4, 0.3, 0.2, 0.1\}$, the individuals are assigned to different operators according to these probabilities for optimization. The numbers of successful and failed optimizations are recorded as S_N^t and F_N^t, respectively, and the current operator ranking R_N^t is computed based on the optimization successes. Both S_N^t and F_N^t are stored in the information aggregation table. Finally, the selection probabilities for the next generation, P_N^{t+1}, are produced via Eqs. (20) and (21). The detailed procedure is as follows.

(1) Initialization: Initialize sub-populations, selection probabilities, and parameters; construct an aggregated information table of length L; and set the prior distribution of β as $\theta_i \sim \text{Beta}(1, 1)$.

(2) Data collection: Aggregate performance data of operators into the table. At the population level, the ranking score is calculated as $r = \frac{\sum s_i}{\sum n_i}$. This reflects the current-stage performance of operators. Once the length of the table exceeds L, earlier records are discarded using a first-in-first-out strategy.

(3) Bayesian updating: Update the posterior distribution of operator success probabilities using the Beta distribution. The posterior expectation is given as $E[\theta_i] = \frac{1+s_i}{2+n_i}$. The posterior mean reflects the optimization performance of the algorithm, where the introduction of a smoothing technique addresses the inefficiency of small samples and prevents excessive optimization of individual algorithms in specific domains.

(4) Weight Assignment: Combine the posterior mean and ranking adjustment to determine the weight of each algorithm based on collective intelligence. The weighting formula is expressed as:

$$w_i = E[\theta_i] \cdot RankFactor_i = \left(\frac{1+s_i}{2+n_i}\right) \cdot \left(1 + \lambda \cdot \frac{N_{ns} + 1 - Rank_i}{N_{ns}}\right) \cdot \left(\frac{S_i}{F_i}\right) \tag{20}$$

where, λ is the ranking adjustment coefficient, N_{ns} denotes the number of algorithms, and higher success rates and ranks increase the weight of the algorithm.

(5) Selection Probability: Normalize the weights to compute the selection probability.

$$P(N_i) = \frac{w_i}{\sum_{j=1}^{N_{ns}} w_j} \tag{21}$$

(6) Reinforced Selection: Apply re-normalized selection probabilities, where probabilities below a fixed threshold $P_{\min} = \frac{1}{2N_{ns}}$ are reassigned. This ensures that each algorithm retains a non-zero selection probability, thus maintaining a balance between exploitation and exploration.

3.9. Energy-saving strategy

Energy-saving strategies constitute an essential component of green intelligent manufacturing. In scheduling environments, reducing idle times between operations is an effective means of saving energy [41].

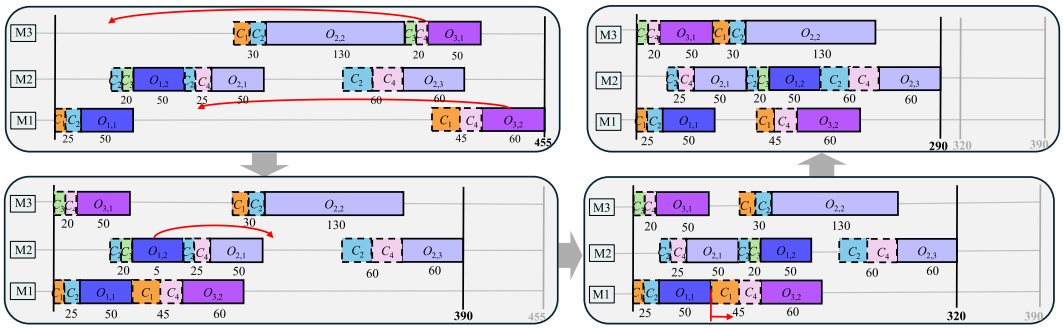


Fig. 8. Example diagram of the energy consumption strategy.

Therefore, under the premise of not increasing the makespan, a delayed full-active scheduling strategy based on heuristic rules is designed.

Heuristic rule: Shutting down machines during idle periods can effectively reduce energy consumption. This rule stipulates that machines can be switched off when the idle time exceeds a predefined safety threshold, thereby saving energy.

Delayed full-active schedule: An active schedule does not allow left-shifting of $O_{i,j}$, while it has been proven that the solution set of a full-active schedule must contain at least one optimal solution [54]. The full-active schedule attempts to right-shift $O_{i,j}$ into idle windows without delaying subsequent operations. However, due to competitive relationships among collaborative units, delaying $O_{i,j}$ may prolong the processing time, thereby aligning the machine's idle window with the heuristic rule and achieving the energy-saving effect. Fig. 8 presents an illustrative example of the proposed strategy. First, the left-to-right detection checks whether there exists a left shift of $O_{i,j}$ that reduces the makespan of an active schedule. Then, the right-to-left detection examines whether there exists an idle window to which $O_{i,j}$ can be right-shifted, further reducing idle energy consumption without increasing the makespan. Through this process, a full-active schedule solution is obtained. Finally, the iteration proceeds by checking whether further delaying $O_{i,j}$ still yields solutions without increasing the makespan, thereby generating the delayed full-active schedule set.

3.10. Complexity of GFLMA

Based on the environmental selection step, the complexity of crossover and mutation operations is $O(G \cdot ps)$, where G denotes the number of generations. The complexity of regional ancestry filtering is $O(G \cdot ps)$. From Algorithm 2, it can be observed that GFLMA mainly consists of the following components: (1) Regional initialization, with a complexity of $O(ps)$. (2) Non-dominated sorting, with a complexity of $O(G \cdot ps^2)$. (3) Energy-saving strategy, with a complexity of $O(G \cdot ps)$. (4) VNS, with a complexity of $O(G \cdot ps)$. (5) WoC-based decision-making, with a complexity of $O(G \cdot N_i)$, where N_i represents the number of operators. In summary, the overall iterative complexity of GFLMA is $O(G \cdot ps^2)$.

4. Experiment and discussion

This section evaluates the effectiveness of the proposed algorithm through detailed numerical experiments, including parameter calibration, ablation studies, and comparative experiments. All experiments were conducted in a Microsoft Windows 11 environment equipped with an Intel Core i9 3.9 GHz CPU, 64 GB of RAM, and an NVIDIA GeForce RTX 4080 GPU. GFLMA and the comparative multi-objective evolutionary algorithms (MOEAs) without deep reinforcement learning (DRL) modules were implemented in Python 3.8, while those integrated with DRL were implemented in Python 3.10. The CUDA version was 12.1, and the PyTorch version was 2.4.0 + cu121. Each algorithm was independently executed 10 times.

Algorithm 2: Main procedure of GFLMA

Input: regional ratio (r_i), population size (ps), crossover rate (p_c), mutation rate (p_m), ranking weight factor (λ), aggregation table length (L)
Output: Pareto solution set (PS)

$P_{regional} \leftarrow$ InitializePopulation(ps, r_i);
Initialize algorithm parameters: number of operators (N_i), selection probability ($P^t_{N_i}$), successes and failures ($S^t_{N_i}, F^t_{N_i}$), initial ranking ($R^t_{N_i}$), aggregation table (T);
$NFEs \leftarrow 0$, $maxNFEs \leftarrow 200 \times ps$, $t \leftarrow 1$;
while $NFEs < maxNFEs$ **do**
  $P_{regional} \leftarrow$ EnvironmentalSelection($P_{regional}, t, p_c, p_m$);
  $Q \leftarrow$ RemoveDuplicates($P_{regional}$);
  $\{Q_0, Q_1, \ldots\} \leftarrow$ FastNonDominatedSorting(Q);
  $i \leftarrow 0$, $NP \leftarrow \emptyset$, $numNP \leftarrow 0$;
  while $numNP + |Q_i| < ps$ **do**
    $NP \leftarrow NP \cup Q_i$;
    $numNP \leftarrow |NP|$;
    $i \leftarrow i + 1$;
  if $numNP < ps$ **then**
    $NP \leftarrow NP \cup$CrowdingDistanceSelection($Q_i, ps - numNP$);
  $NFEs \leftarrow NFEs + ps$;
  $P_{regional} \leftarrow$ EnergySavingStrategy(NP);
  $spPool \leftarrow$ RouletteSelection($P_{regional}, N_i, P^t_{N_i}$);
  $(P_{regional}, S^t_{N_i}, F^t_{N_i}, R^t_{N_i}) \leftarrow$ VNS($spPool$);
  if $t > L$ **then**
    $T \leftarrow$ RemoveHistoricalRecords(T, L);
  $(P^t_{N_i}, T, R^t_{N_i}) \leftarrow$ WoC-DecisionMaking($\lambda, T, S^t_{N_i}, F^t_{N_i}, R^t_{N_i}$);
  $t \leftarrow t + 1$;
$PS \leftarrow$ FastNonDominatedSorting($P_{regional}$);

4.1. Experimental instances and metrics

Since the CHFJS problem has not been previously investigated, 30 instances of varying scales were generated to validate the effectiveness of GFLMA. The instances can be accessed via the provided link.[1] The number of jobs was set to $n \in \{10, 15, 20, 30, 50, 100\}$, the number of machines to $m \in \{5, 10, 20, 30, 50\}$, and the number of collaborative units to $r \in \{6, 10, 15, 20, 40\}$. Each job contained $J_i \in [3, 7]$ operations, with processing times $T_{i,j,k} \in [10, 20]$ and collaborative unit times $T_{t,i,j,k} \in [2, 5]$. The number of eligible machines for each operation was $M_{i,j} \in [2, 50]$, ensuring that $m > M_{i,j}$ in the same instance.

[1] https://github.com/qhqye/CHFJS-Instance

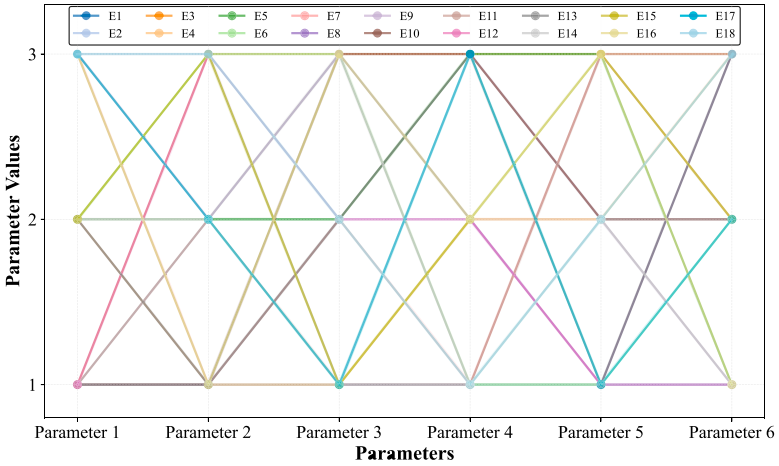


Fig. 9. Parameter combinations.

The processing power, idle power, power consumption of collaborative units, and restart energy consumption are set as $N_P = 5.0$ kW, $N_I = 1.0$ kW, $N_C = 2.0$ kW, and $E_R = 5.0$ kWh, respectively.

A combination of performance metrics was employed to assess the algorithms from multiple perspectives. Specifically, the hypervolume (HV) [55], inverted generational distance (IGD) [56], and the distribution uniformity (Spread) metric [45] were adopted, and their detailed definitions are provided in the supplementary material. A larger HV value indicates better overall performance, whereas smaller IGD and Spread values indicate better convergence and distribution uniformity, respectively. Since the true Pareto front (PF) of CHFJS is unknown, we followed the widely used approximation strategy in [4]. The reference point for HV was set to $[1.2, 1.2]$. For IGD, the reference PF was constructed by collecting all solutions obtained from all algorithms across all independent runs and then extracting the non-dominated solutions from this aggregated set. This procedure provides a fair benchmark for evaluating convergence and diversity without relying on any externally known optimal solution set.

4.2. Parameter calibration

To systematically analyze the impact of parameters on system performance, an experimental scheme based on orthogonal design was adopted. Meanwhile, to avoid overfitting, an independent calibration dataset containing six instances of different sizes was generated (Table S-4 in supplementary materials). The GFLMA involves six key parameters: (1) population size ps, (2) crossover rate p_c, (3) mutation rate p_m, (4) ranking weight factor λ, (5) aggregation table length L, and (6) regional ratio r_i. In this section, a partial orthogonal design from the Taguchi method was employed to construct 18 experimental groups (E1–E18), allowing for a comprehensive investigation of the influence of the six parameters within a limited number of experiments. Each parameter was set to three levels: ps = [200,240,280], p_c = [0.8,0.9,1.0], p_m = [0.1,0.15,0.2], λ = [0.4,0.5,0.6], L = [20,30,40], r_i = {[0.4,0.4,0.1,0.1], [0.3,0.3,0.2,0.2], [0.25,0.25,0.25,0.25]}. For fairness, all algorithms were executed under the same stopping condition, $\max NFE_s = 200 \times \sum_{i=1}^{ps} n_i$. The detailed parameter combinations and experimental design are illustrated in Fig. 9.

The influence trends of the six key parameters on the three primary performance metrics are illustrated in Fig. 10. Variations in parameter levels exhibited distinct effects on each metric, among which p_c, L, and r_i showed particularly significant impacts on IGD. Based on the main effects analysis, the optimal parameter configuration was determined as [2, 1, 2, 3, 3, 2], where $ps = 240$, $p_c = 0.8$, $p_m = 0.15$, $\lambda = 0.6$, $L = 40$, and r_i = [0.3,0.3,0.2,0.2]. Under this configuration, an effective balance was achieved among inherently conflicting objectives, including convergence, diversity, and distribution quality.

Table 2
Comparative results of OR-Tools and GFLMA under different instance sizes.

Instance size	OR-Tools			GFLMA		
	Makespan	TEC	t(s)	Makespan	TEC	t(s)
3 × 2 × 2	172	2423	4.07	176	1866	212.41
5 × 3 × 3	153	2890	1800	162	2103	325.86
10 × 5 × 3	247	5271	1800	263	3792	382.09
15 × 10 × 6	–	–	–	242	4947	412.77
20 × 10 × 8	–	–	–	359	6171	642.32

The shutdown threshold T_{th} is a core parameter for energy consumption calculations. Therefore, given the setting of $T_{t,i,j,k} \in [2, 5]$, a sensitivity analysis and parameter calibration were conducted under different thresholds $T_{th} \in \{10, 15, 20\}$. The results in Fig. 11 indicate that as T_{th} increases from 10 to 20, the accumulation of excess energy during unproductive idling leads to a distinct upward trend in TEC. Simultaneously, the IGD and HV metrics, which represent the quality of the solution set, also deteriorate. This demonstrates that a smaller threshold more effectively reduces non-productive waiting and outperforms the strategy of extending waiting times. Consequently, $T_{th} = 10$ is adopted for all subsequent experiments.

4.3. Model validation

To verify the correctness of the proposed MILP model, the Google OR-Tools solver was applied to representative instances of different scales, and its results were compared with those of GFLMA. Since exact solvers are typically used for single-objective optimization, the bi-objective model was converted into a single-objective formulation using the widely adopted weighted-sum method [24]. Specifically, the weight coefficient of TEC was set to 0, so that makespan minimization became the optimization objective. The solver was then invoked to obtain a schedule, and the corresponding TEC value was computed based on this schedule. Under these settings, five representative instances of small and medium scales were designed for testing, and the time limit of OR-Tools was set to 1800 s. The experimental results are reported in Table 2.

The results show that, for the 3 × 2 × 2 instance, OR-Tools can obtain the optimal solution within a short time, and the makespan achieved by GFLMA is very close to it. As the instance scale increases, the computational burden of the exact solver rises substantially. For the

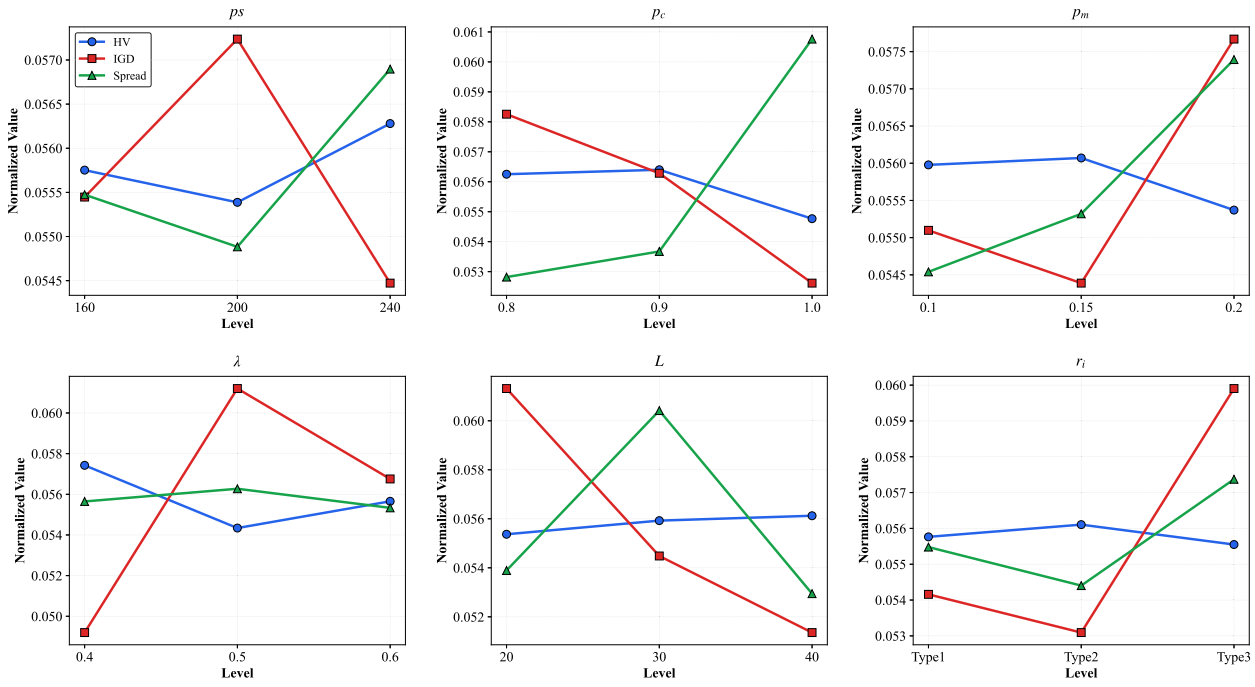


Fig. 10. Influence of key parameters on algorithm performance.

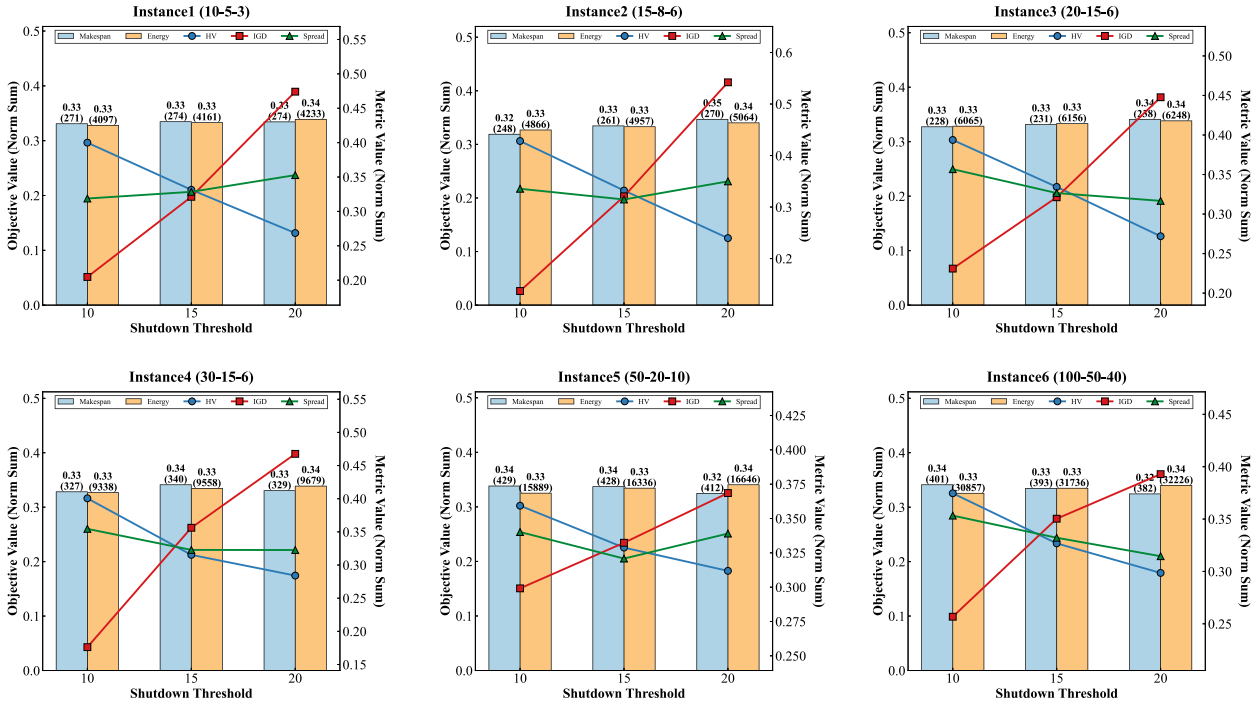


Fig. 11. Sensitivity analysis and parameter calibration under different shutdown thresholds T_{th}.

5 × 3 × 3 and 10 × 5 × 3 instances, OR-Tools can only return feasible solutions within the time limit and cannot guarantee optimality. When the scale further increases to 15 × 8 × 6, OR-Tools is unable to produce a feasible solution within the time limit. In contrast, although GFLMA cannot always attain the optimal makespan for every instance, it achieves competitive solutions for all tested scales. Therefore, since the true PF can only be obtained for small scale instances via exact solving, the reference PF in the subsequent experiments is constructed using the widely used strategy of merging non-dominated solution sets [2].

4.4. Ablation study

To evaluate the effectiveness of each component in GFLMA, five variant algorithms were designed for comparison. The specific modifications are as follows: (1) GFLA, a GFLMA variant that retains only the regional initialization component, used for baseline comparison; (2) GFLMA-VNS, a GFLMA variant without the VNS component, designed to demonstrate the necessity of the VNS design; (3) GFLMA-TES, a variant based on non-dominated selection, constructed to investigate the effectiveness of intergenerational environmental selection; (4)

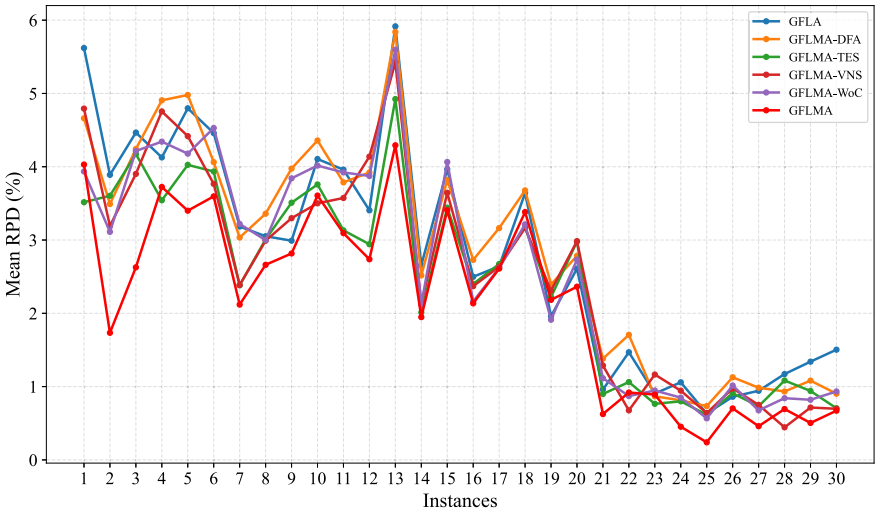


Fig. 12. Energy consumption comparison of all instances.

Table 3
Friedman test comparative results of the variant algorithms (significance level $\alpha = 0.05$).

MOEAs	HV		IGD		Spread	
	rank	*p*-value	rank	*p*-value	rank	*p*-value
GFLA	5.00		4.87		**2.23**	
GFLMA-VNS	3.57		3.70		4.30	
GFLMA-TES	2.93	2.63E−12	3.10	1.95E−11	3.40	7.04E−6
GFLMA-WoC	3.67		3.63		3.00	
GFLMA-DFA	4.13		4.07		4.77	
GFLMA	**1.70**		**1.63**		3.20	

GFLMA-WoC, a variant that removes the WoC-based operator selection to validate its contribution; and (5) GFLMA-DFA, a variant without the energy-saving strategy, highlighting the necessity of the proposed energy-saving mechanism. Finally, all these variants are compared with the complete GFLMA.

The detailed results of HV, IGD, and Spread are reported in Tables S-5 to S-7 of the supplementary material, with the best results highlighted in bold. In terms of mean HV values, GFLMA exhibits a pronounced advantage over the compared algorithms. When considering the best HV values, GFLA, GFLMA-VNS, GFLMA-WoC, and GFLMA-DFA all perform significantly worse than GFLMA, whereas GFLMA-TES achieves the best results on five instances. This validates the effectiveness of the fundamental components, including neighborhood structures, the WoC-based decision model, and the energy-saving strategy. Moreover, a further comparison with GFLMA-TES indicates that GFLMA performs better on a larger number of instances, suggesting that incorporating intergenerational environmental selection can generate a solution set closer to the PF, thereby effectively enhancing the overall performance of the algorithm. The observations for the IGD metric are consistent with those for HV. Regarding the Spread metric, the variants without the WoC operator selection model (GFLA and GFLMA-WoC) exhibited better solution-set diversity. This reveals that the WoC decision mechanism dynamically prioritizes operators with high success rates through Bayesian inference, thereby strengthening the exploitation capability of the algorithm. Although this strategy accelerates population convergence toward the Pareto front and improves HV and IGD performance, it also causes the search process to concentrate on specific regions that can rapidly enhance solution quality, reducing random exploration in sparsely populated areas of the solution space. The degradation of GFLMA in terms of the Spread metric essentially reflects a trade-off made to pursue higher convergence accuracy. Therefore, this result is reasonable.

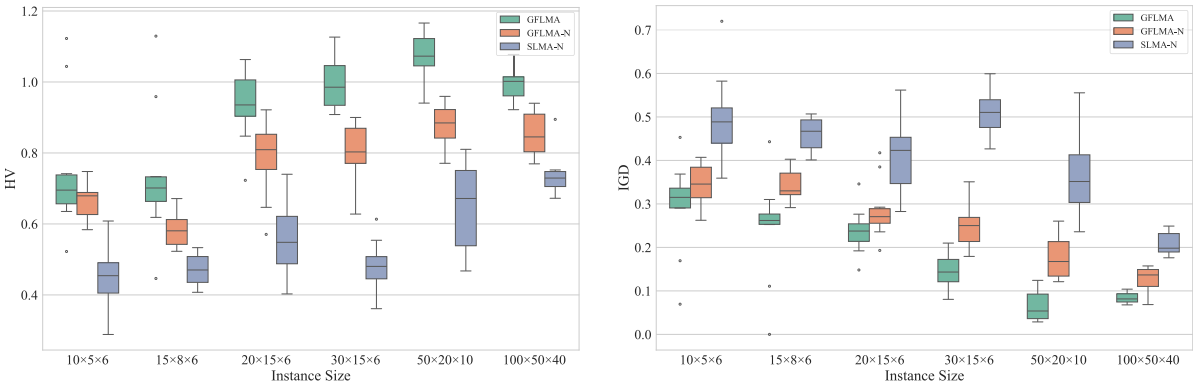
Table 3 presents the Friedman test results of the six MOEAs with respect to HV, IGD, and Spread. The results indicate that GFLMA achieved the lowest mean ranks for HV (1.70) and IGD (1.63), highlighting its significant advantage in solution quality. For all three metrics, the p-values were much smaller than 0.05, confirming the statistical significance of the performance differences among the algorithms. GFLA and GFLMA-WoC obtained the best and second-best mean ranks (2.23 and 3.00, respectively) for the Spread metric, further confirming that the WoC-based operator selection model improves solution quality at the expense of some diversity.

To evaluate the energy-saving optimization capability of GFLMA, the relative percentage deviation (RPD) of energy consumption is reported for all algorithms across different instances. For a given instance, the RPD is calculated as RPD $= (TEC_{\text{avg}} - TEC_{\text{min}})/TEC_{\text{min}}$, where TEC_{avg} denotes the average total energy consumption obtained over multiple runs on the considered instance, and TEC_{min} denotes the globally best (minimum) energy consumption value found by all compared algorithms across all runs on that instance. The experimental results are presented in Fig. 12.

The results indicate that for instances of small to medium size (Instances 1–15), GFLMA-DFA (without the energy-saving strategy) and the baseline GFLA exhibit consistently higher RPD values, indicating a clear performance disadvantage. In particular, the GFLMA-DFA curve shows pronounced fluctuations, suggesting that without the proposed energy-saving decoding strategy, both convergence stability and energy optimization capability deteriorate significantly. As the instance size increases (Instances 16–30), the larger magnitude of the reference optimal value TEC_{min} naturally narrows the relative deviations, resulting in a downward trend in the overall RPD curves. Nevertheless, the relative rankings and performance gaps among the algorithms remain largely consistent. This implies that the advantage provided by the energy-saving strategy is not an incidental effect of scale but demonstrates desirable robustness and transferability. Furthermore, the results of the GFLMA indicate that the synergistic combination of all components substantially enhances overall performance, achieving the best or near-best TEC on most instances. This validates the effectiveness of each component and the necessity of the proposed integrated design.

4.5. Comparison and discussions

To evaluate the performance of GFLMA in solving the CHFJS problem, comparisons were conducted with the mainstream algorithm NSGA-II [45]. In addition, the benchmark algorithms included the



(a) HV metric (b) IGD metric

Fig. 13. Performance distribution under different shutdown rules with collaborative unit occupancy constraints.

Table 4
Friedman test on the effect of VNS on comparative baselines (significance level $\alpha = 0.05$).

MOEAs	HV		IGD		Spread	
	rank	*p*-value	rank	*p*-value	rank	*p*-value
LMPFE	6.00	7.73E−28	6.00	1.45E−26	3.73	5.55E−07
LMPFE+VNS	5.00		5.00		4.43	
MOEA/D-DDWA	3.97		3.77		3.83	
MOEA/D-DDWA+VNS	2.63		2.67		4.27	
NSGA-II	2.00		2.03		2.63	
NSGA-II+VNS	**1.40**		**1.53**		**2.10**	

Table 5
Friedman test results of all comparative algorithms (significance level $\alpha = 0.05$).

MOEAs	HV		IGD		Spread	
	rank	*p*-value	rank	*p*-value	rank	*p*-value
NSGA-II	4.30	5.20E−26	3.97	1.52E−23	3.67	1.25E−1
DQCE	3.40		3.07		3.73	
LMPFE	6.00		6.00		3.10	
MOEA/D-DDWA	4.10		4.37		**2.80**	
SLMA	2.10		2.27		3.97	
GFLMA	**1.10**		**1.33**		3.73	

reinforcement learning-based DQCE [29] and the heuristic MOEA/D-DDWA [41] for heterogeneous flexible job shop scheduling, as well as the recently proposed reinforcement learning-based SLMA [10] and the heuristic LMPFE [57]. For fairness, some parameters of the comparative algorithms were aligned, such as $ps = 240$, $p_c = 0.8$, and $p_m = 0.15$, while other parameters followed the original configurations of the respective algorithms. Specifically, the parameter settings were as follows: for DQCE, learning rate $\alpha = 0.001$, batch size $bs = 64$, greedy factor $\epsilon = 0.9$, discount factor $\gamma = 0.9$, and experience pool size $S_E = 512$; for SLMA, $\alpha = 0.1$, $\epsilon = 0.85$ and $\gamma = 0.8$; for MOEA/D-DDWA, neighborhood size $T = 10$, and weight adjustment $w_a = 1$; and for LMPFE, the frequency of employing generic front modeling $f_{PFE} = 0.1$, and number of subregions $K = 5$. For a fair comparison, all comparative MOEAs were equipped with regional initialization and VNS. The regional initialization strategy has been verified not to introduce systematic bias. However, whether different algorithms benefit from the additional VNS to a similar extent still needs to be further examined through control experiments.

Therefore, for comparative MOEAs whose original versions do not include VNS, we conducted independent control experiments. Specifically, NSGA-II, MOEA/D-DDWA, and LMPFE were tested under the original configuration and under an enhanced configuration equipped with VNS. These experiments aim to analyze the impact of VNS on the search behavior and performance of different algorithms. The Friedman test results are reported in Table 4. The more detailed results are provided in the supplementary material in Tables S-8 to S-11. After incorporating VNS, all algorithms achieve improvements to some extent in terms of convergence performance and solution-set quality. Notably, the improvement exhibits a consistent trend across different algorithms and does not change their relative performance ranking. This indicates that VNS mainly serves to uniformly raise the baseline search quality rather than providing targeted reinforcement for a specific algorithm. Therefore, introducing VNS does not bring systematic bias to the fair comparison among algorithms.

The detailed metric results for all comparative algorithms are reported in Tables S-12 to S-14. From the comparative results, GFLMA achieves the best and average HV values on 27 instances, and its overall mean performance is superior to that of the comparative algorithms. For the IGD metric, GFLMA obtained the best solutions in 24 instances (best) and 23 instances (average). Summarizing across these two metrics, GFLMA outperformed state-of-the-art MOEAs in at least 70% of the instances. Regarding the Spread metric, GFLMA did not exhibit a clear advantage. This indicates that GFLMA's strength lies in its strong local search capability based on global feedback, which prioritizes eliminating individuals that contribute to distribution breadth but suffer from poor convergence. In contrast, the comparative algorithms maintain higher Spread values by retaining more boundary solutions, resulting in deficiencies in their HV and IGD performance. The Friedman test results for all algorithms across all instances are reported in Table 5. GFLMA ranked first in both HV and IGD, with *p*-values much smaller than 0.05, demonstrating significant differences compared with other algorithms. Although MOEA/D-DDWA achieved the highest rank in Spread, the ranking differences among the algorithms were minor, suggesting that the loss in distribution quality for GFLMA is acceptable. Moreover, the Friedman test on Spread yields $p > 0.05$, indicating no significant differences among the algorithms on this metric.

The preceding experiments have validated the superiority of GFLMA. However, in practical collaborative environments, machine shutdown decisions are often constrained by the occupancy status of collaborative units. To further examine the robustness of the algorithm under this condition, a new shutdown rule that considers collaborative unit occupancy is introduced and analyzed. Specifically, the start time of the next operation, $S_{p+1,k}$, in Eqs. (13) and (14) of the MILP model is replaced by the start time of the required collaborative unit C^2. Based on six representative instances of varying scales, GFLMA-N (under the new rule) is compared with GFLMA (under the original rule) and SLMA-N (under the new rule). The results are presented in Fig. 13. The results indicate that the collaborative constraint narrows the feasible solution space, leading to a slight

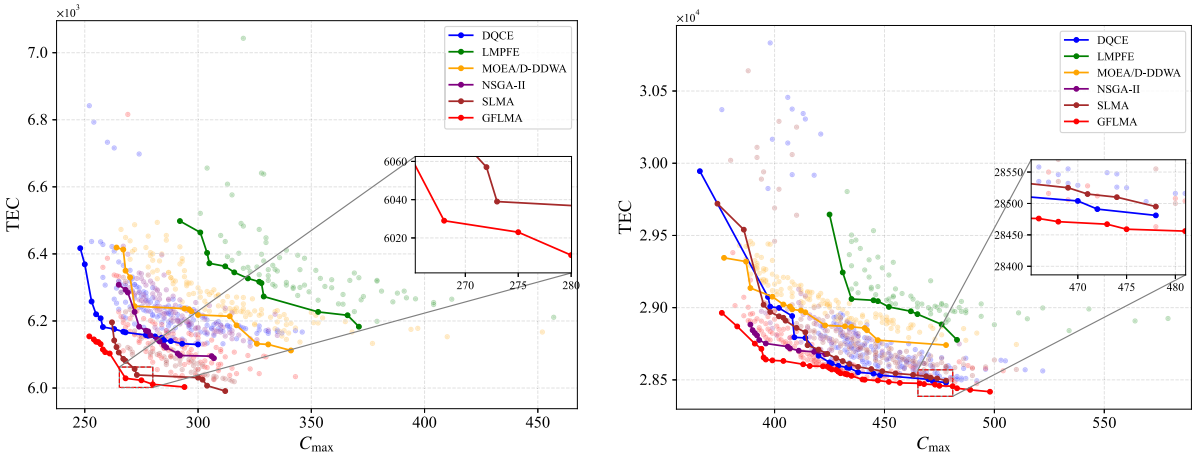


Fig. 14. Example of the PFs.

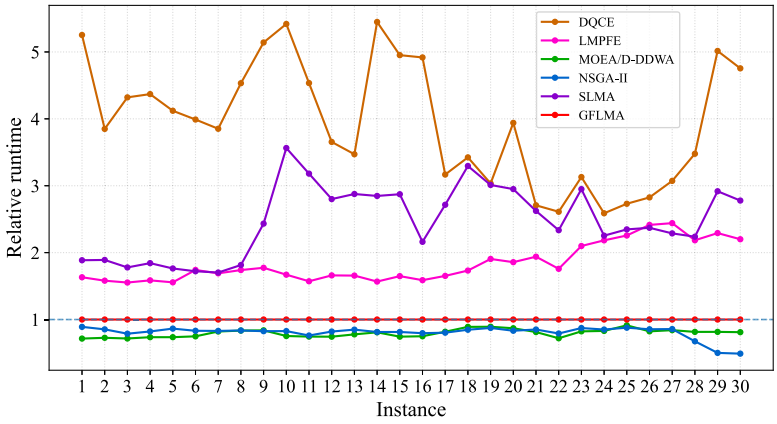


Fig. 15. The relative running time of all instances.

performance degradation of GFLMA-N compared with GFLMA, which is consistent with expectations. Nevertheless, under the same constraint, GFLMA-N still significantly outperforms SLMA-N, demonstrating that the proposed method maintains strong robustness and adaptability under more stringent collaborative restrictions.

The success of GFLMA can be attributed to its framework design. First, regional initialization ensures a diverse selection of solutions. Second, intergenerational environmental selection guided by feedback enables the learning and estimation of the probabilities of solution quality. Third, twelve problem-specific neighborhood structures enhance convergence. Fourth, the WoC decision-making model adaptively updates the selection probability of operators based on performance feedback. Finally, the delayed scheduling strategy continues optimizing TEC even after obtaining an optimal scheduling solution.

Several figures provide more comprehensive comparisons of the performance of all MOEAs. Fig. 14 illustrates the complete set of scheduling solutions obtained over multiple runs on medium- and large-scale instances, where the solid lines represent the overall PFs. It can be concluded that GFLMA consistently achieved the best overall PF, and the majority of its non-PF solutions still outperformed those of the comparative algorithms. Fig. 15 presents the average runtime of all MOEAs across all instances, normalized against GFLMA. DQCE incurred additional computational overhead due to the integration of q-learning during the training process. Although GFLMA is not the most computationally efficient, its moderate runtime overhead is justified by the substantial improvement in solution quality.

5. Conclusion

This paper investigated the flexible job shop scheduling problem with human–robot collaboration under the context of Industry 5.0. To achieve rational allocation of human and robot resources, a memetic algorithm integrated with a feedback learning mechanism, termed GFLMA, was proposed. First, a global feedback learning-based regional initialization method was introduced, which assigns regional lineage to individuals and enlarges the search space. Experimental results verified the effectiveness of this regional initialization. Second, an intergenerational environmental selection mechanism was designed to select individuals through adaptive feedback and learning. A VNS strategy incorporating 12 problem-specific neighborhood structures was developed, and a WoC model was adopted to learn operator selection. Finally, an energy-saving strategy was tailored for the CHFJS. Computational experiments on CHFJS instances of varying scales demonstrated that GFLMA achieved the best HV performance in more than 70% of instances and outperformed comparative algorithms in terms of IGD in over 85% of instances, thereby validating the effectiveness of the proposed approach.

The design principles of GFLMA provide a valuable reference for addressing complex application problems and enhancing the adaptability of algorithms. Nevertheless, additional production factors must be considered in real-world applications. Future research will therefore focus on the following directions: (1) incorporating more optimization objectives; (2) accounting for the dominant role of human workers in

human–robot collaboration; and (3) modeling scenarios where workers are affected by fatigue in collaborative environments.

CRediT authorship contribution statement

Hongquan Qu: Writing – original draft, Data curation. **Shiliang Shao:** Visualization, Validation. **Yunhong Xu:** Software, Investigation. **Maolin Cai:** Writing – review & editing. **Yan Shi:** Formal analysis. **Xiaomeng Tong:** Supervision, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (Grant No. 52575046), the National Key Research and Development Program of China (Grant No. 2024YFB3409700), and the Beijing New Generation Information and Communication Technology Innovation Special Program (Grant No. Z251100008125039).

Appendix A. Supplementary data

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.swevo.2026.102392.

Data availability

Data will be made available on request.

References

[1] X. Chen, J. Li, Z. Wang, Q. Chen, K. Gao, Q. Pan, Optimizing dynamic flexible job shop scheduling using an evolutionary multi-task optimization framework and genetic programming, IEEE Trans. Evol. Comput. (2025).
[2] Z. Yang, X. Hu, Y. Li, M. Liang, K. Wang, L. Wang, H. Tang, S. Guo, A Q-learning-based improved multi-objective genetic algorithm for solving distributed heterogeneous assembly flexible job shop scheduling problems with transfers, J. Manuf. Syst. 79 (2025) 398–418.
[3] J. Li, X. Li, L. Gao, Q. Liu, A flexible job shop scheduling problem considering on-site machining fixtures: A case study from customized manufacturing enterprise, IEEE Trans. Autom. Sci. Eng. (2024).
[4] D. Wang, J. Zhang, Flow shop scheduling with human–robot collaboration: A joint chance-constrained programming approach, Int. J. Prod. Res. 62 (4) (2024) 1297–1317.
[5] K. Arviv, H. Stern, Y. Edan, Collaborative reinforcement learning for a two-robot job transfer flow-shop scheduling problem, Int. J. Prod. Res. 54 (4) (2016) 1196–1209.
[6] A.R. Sadik, B. Urban, Flow shop scheduling problem and solution in cooperative robotics—Case-study: One cobot in cooperation with one worker, Futur. Internet 9 (3) (2017) 48.
[7] A. Kinast, R. Braune, K.F. Doerner, S. Rinderle-Ma, C. Weckenborg, A hybrid metaheuristic solution approach for the cobot assignment and job shop scheduling problem, J. Ind. Inf. Integr. 28 (2022) 100350.
[8] B.L. Vermin, D. Abbink, F. Schulte, Bi-objective job-shop scheduling considering human fatigue in cobotic order picking systems: A case of an online grocer, Procedia Comput. Sci. 232 (2024) 635–644.
[9] C. Lu, R. Gao, L. Yin, B. Zhang, Human–robot collaborative scheduling in energy-efficient welding shop, IEEE Trans. Ind. Inform. 20 (1) (2024) 963–971.
[10] F. Yu, C. Lu, L. Yin, B. Zhang, A self-learning memetic algorithm for human-robot collaboration scheduling in energy-efficient distributed mixed fuzzy welding shop, IEEE Trans. Autom. Sci. Eng. 22 (2025) 6595–6607.
[11] H. Bao, Q. Pan, C.-M. Chew, L. Wang, L. Gao, Energy-efficient distributed heterogeneous hybrid flow-shop scheduling using graph neural network and deep reinforcement learning, IEEE Trans. Autom. Sci. Eng. (2025).
[12] L.V. Ho, T. Bui-Tien, M.A. Wahab, A two-step failure identification approach using a stochastic optimization-based ensemble learning model for beams without pristine data, Eng. Struct. 334 (2025) 120253.
[13] F. Yu, C. Lu, J. Zhou, L. Yin, Mathematical model and knowledge-based iterated greedy algorithm for distributed assembly hybrid flow shop scheduling problem with dual-resource constraints, Expert Syst. Appl. 239 (2024) 122434.
[14] A. García, Greedy algorithms: A review and open problems, J. Inequal. Appl. 2025 (1) (2025) 11.
[15] M. Thenarasu, K. Rameshkumar, J. Rousseau, S. Anbuudayasankar, Development and analysis of priority decision rules using MCDM approach for a flexible job shop scheduling: A simulation study, Simul. Model. Pr. Theory 114 (2022) 102416.
[16] J. Bai, N.V. Nguyen, H. Nguyen-Xuan, G. Kosec, L. Wang, M.A. Wahab, A multi-objective optimization of porous sandwich functionally graded plates with graphene nanoplatelet reinforcement using blood-sucking leech optimizer, Compos. Struct. 357 (2025) 118921.
[17] J. Xie, X. Li, L. Gao, L. Gui, A hybrid genetic tabu search algorithm for distributed flexible job shop scheduling problems, J. Manuf. Syst. 71 (2023) 82–94.
[18] Y. Xu, D. Wang, M. Zhang, M. Yang, C. Liang, Quantum particle swarm optimization with chaotic encoding schemes for flexible job-shop scheduling problem, Swarm Evol. Comput. 93 (2025) 101836.
[19] J. Bai, H. Nguyen-Xuan, E. Atroshchenko, G. Kosec, L. Wang, M.A. Wahab, Blood-sucking leech optimizer, Adv. Eng. Softw. 195 (2024) 103696.
[20] R. Xu, Y. Xu, W. Xiao, A Q-learning based memetic algorithm for flexible flow shop scheduling with active waiting and job switching, Swarm Evol. Comput. 100 (2026) 102227.
[21] C. Su, C. Zhang, C. Wang, W. Cen, G. Chen, L. Xie, Fast Pareto set approximation for multi-objective flexible job shop scheduling via parallel preference-conditioned graph reinforcement learning, Swarm Evol. Comput. 88 (2024) 101605.
[22] C. Lu, Y. Huang, L. Meng, L. Gao, B. Zhang, J. Zhou, A Pareto-based collaborative multi-objective optimization algorithm for energy-efficient scheduling of distributed permutation flow-shop with limited buffers, Robot. Comput.-Integr. Manuf. 74 (2022) 102277.
[23] R. Li, W. Gong, L. Wang, C. Lu, X. Zhuang, Surprisingly popular-based adaptive memetic algorithm for energy-efficient distributed flexible job shop scheduling, IEEE Trans. Cybern. (2023) 1–11.
[24] F. Yu, C. Lu, J. Zhou, L. Yin, K. Wang, A knowledge-guided bi-population evolutionary algorithm for energy-efficient scheduling of distributed flexible job shop problem, Eng. Appl. Artif. Intell. 128 (2024) 107458.
[25] X. Chang, X. Jia, J. Ren, A reinforcement learning enhanced memetic algorithm for multi-objective flexible job shop scheduling toward industry 5.0, Int. J. Prod. Res. 63 (1) (2025) 119–147.
[26] K. Huang, R. Li, W. Gong, R. Wang, H. Wei, BRCE: Bi-roles co-evolution for energy-efficient distributed heterogeneous permutation flow shop scheduling with flexible machine speed, Complex Intell. Syst. 9 (5) (2023) 4805–4816.
[27] J. Yang, H. Xu, J. Cheng, R. Li, Y. Gu, A decomposition-based memetic algorithm to solve the biobjective green flexible job shop scheduling problem with interval type-2 fuzzy processing time, Comput. Ind. Eng. 183 (2023) 109513.
[28] K. Huang, W. Gong, C. Lu, An enhanced memetic algorithm with hierarchical heuristic neighborhood search for type-2 green fuzzy flexible job shop scheduling, Eng. Appl. Artif. Intell. 130 (2024) 107762.
[29] R. Li, W. Gong, L. Wang, C. Lu, C. Dong, Co-evolution with deep reinforcement learning for energy-aware distributed heterogeneous flexible job shop scheduling, IEEE Trans. Syst. Man Cybern.: Syst. 54 (1) (2023) 201–211.
[30] L. Nguyen-Ngoc, Q. Nguyen-Huu, G. De Roeck, T. Bui-Tien, M. Abdel-Wahab, Deep neural network and evolved optimization algorithm for damage assessment in a truss bridge, Mathematics 12 (15) (2024) 2300.
[31] J. McCoy, D. Prelec, A Bayesian hierarchical model of crowd wisdom based on predicting opinions of others, Manag. Sci. 70 (9) (2024) 5931–5948.
[32] M. Boon, Predicting elections: A 'wisdom of crowds' approach, Int. J. Mark. Res. (2012).
[33] W. Kristjanpoller, K. Michell, M.C. Minutolo, P. Dheeriya, Trading support system for portfolio construction using wisdom of artificial crowds and evolutionary computation, Expert Syst. Appl. 177 (2021) 114943.
[34] C. Zuheros, E. Martínez-Cámara, E. Herrera-Viedma, F. Herrera, Crowd decision making: Sparse representation guided by sentiment analysis for leveraging the wisdom of the crowd, IEEE Trans. Syst. Man Cybern.: Syst. 53 (1) (2023) 369–379.
[35] D. Marbach, J.C. Costello, R. Küffner, N.M. Vega, R.J. Prill, D.M. Camacho, K.R. Allison, M. Kellis, J.J. Collins, G. Stolovitzky, Wisdom of crowds for robust gene network inference, Nature Methods 9 (8) (2012) 796–804.
[36] R.V. Yampolskiy, A. EL-Barkouky, Wisdom of artificial crowds algorithm for solving NP-hard problems, Int. J. Bio-Inspired Comput. 3 (6) (2011) 358–369.
[37] J. Redding, J. Schreiver, C. Shrum, A. Lauf, R. Yampolskiy, Solving NP-hard number matrix games with wisdom of artificial crowds, in: 2015 Computer Games: AI, Animation, Mobile, Multimedia, Educational and Serious Games, CGAMES, 2015, pp. 38–43.
[38] Y. Yoo, A.R. Escobedo, R. Kemmer, E. Chiou, Elicitation and aggregation of multimodal estimates improve wisdom of crowd effects on ordering tasks, Sci. Rep. 14 (1) (2024) 2640.
[39] W. Wang, G. Tian, H. Zhang, Z. Li, L. Zhang, A hybrid genetic algorithm with multiple decoding methods for energy-aware remanufacturing system scheduling problem, Robot. Comput.-Integr. Manuf. 81 (2023) 102509.

[40] Z. Huang, Y. Mei, F. Zhang, M. Zhang, Multitask linear genetic programming with shared individuals and its application to dynamic job shop scheduling, IEEE Trans. Evol. Comput. 28 (6) (2024) 1546–1560.
[41] H. Qu, X. Tong, M. Cai, Y. Shi, X. Lan, Energy-saving scheduling strategy for variable-speed flexible job-shop problem considering operation-dependent energy consumption, Expert Syst. Appl. 256 (2024) 124952.
[42] P. Brandimarte, Routing and scheduling in a flexible job shop by tabu search, Ann. Oper. Res. 41 (3) (1993) 157–183.
[43] S. Dauzère-Pérès, J. Paulli, An integrated approach for modeling and solving the general multiprocessor job-shop scheduling problem using tabu search, Ann. Oper. Res. 70 (1997) 281–306.
[44] Q. Zhang, H. Li, MOEA/D: A multiobjective evolutionary algorithm based on decomposition, IEEE Trans. Evol. Comput. 11 (6) (2007) 712–731.
[45] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, A fast and elitist multiobjective genetic algorithm: NSGA-II, IEEE Trans. Evol. Comput. 6 (2) (2002) 182–197.
[46] Z.Y. Bo, Q. Yang, Y.H. Jia, X.D. Gao, Z.Y. Lu, J. Zhang, Evolutionary algorithm with cross-generation environmental selection for traveling salesman problem, in: Proceedings of the Genetic and Evolutionary Computation Conference Companion, in: GECCO '24 Companion, Association for Computing Machinery, New York, NY, USA, 2024, pp. 635–638.
[47] L. Berterottière, S. Dauzère-Pérès, C. Yugma, Flexible job-shop scheduling with transportation resources, European J. Oper. Res. 312 (3) (2024) 890–909.
[48] R. Li, L. Wang, H. Sang, L. Yao, L. Pan, LLM-assisted automatic memetic algorithm for lot-streaming hybrid job shop scheduling with variable sublots, IEEE Trans. Evol. Comput. (2025) 1.
[49] M. Ziaee, J. Mortazavi, M. Amra, Flexible job shop scheduling problem considering machine and order acceptance, transportation costs, and setup times, Soft Comput. (2022) 1–17.
[50] E. Nowicki, C. Smutnicki, A fast taboo search algorithm for the job shop problem, Manag. Sci. 42 (6) (1996) 797–813.
[51] J. Xie, X. Li, L. Gao, L. Gui, A new neighbourhood structure for job shop scheduling problems, Int. J. Prod. Res. 61 (7) (2023) 2147–2161.
[52] U. Hamid, J. Bai, G. Kosec, L. Wang, M.A. Wahab, Optimization of auxetic honeycomb structures for mechanical performance, Compos. Struct. (2025) 119458.
[53] R. Li, W. Gong, C. Lu, Self-adaptive multi-objective evolutionary algorithm for flexible job shop scheduling with fuzzy processing time, Comput. Ind. Eng. 168 (2022) 108099.
[54] A. Neumann, A. Hajji, M. Rekik, R. Pellerin, Genetic algorithms for planning and scheduling engineer-to-order production: A systematic review, Int. J. Prod. Res. 62 (8) (2024) 2888–2917.
[55] L. While, P. Hingston, L. Barone, S. Huband, A faster algorithm for calculating hypervolume, IEEE Trans. Evol. Comput. 10 (1) (2006) 29–38.
[56] E. Zitzler, K. Deb, L. Thiele, Comparison of multiobjective evolutionary algorithms: Empirical results, Evol. Comput. 8 (2) (2000) 173–195.
[57] Y. Tian, L. Si, X. Zhang, K.C. Tan, Y. Jin, Local model-based Pareto front estimation for multiobjective optimization, IEEE Trans. Syst. Man Cybern.: Syst. 53 (1) (2022) 623–634.